RBE 500 — Foundations of Robotics

Project Assignment 1

Worcester Polytechnic Institute

Tracy Yang Kyle Yee

June 29, 2026

1 Create the Robot

1.1 Overview

A 3-DOF SCARA (Selective Compliance Assembly Robot Arm) manipulator was created in ROS2 and simulated in Gazebo Harmonic (Jazzy). The robot consists of:

- **Joint 1** (θ_1): Revolute joint, rotation about the z-axis, range $[-\frac{\pi}{2}, \frac{\pi}{2}]$
- **Joint 2** (θ_2): Revolute joint, rotation about the z-axis, range $[-\frac{\pi}{2}, \frac{\pi}{2}]$
- **Joint 3** (d_3): Prismatic joint, translation along the z-axis, range $[-0.2, 0.2]$ m

Links are represented with cylinders and boxes. The URDF was written entirely by hand and includes inertial, visual, and collision properties for each link.

1.2 DH Parameters

The robot kinematic parameters used throughout this assignment are summarized in Table 1.

Table 1: SCARA Robot Parameters

Parameter	**Symbol**	**Value**
Column height (Joint 1 z-offset)	l_1	0.45 m
Arm 1 length	a_1	0.40 m
Link 2 length	a_2	0.30 m
Joint 3 range	d_3	$[-0.2, 0.2]$ m

1.3 Robot in Gazebo

The robot was spawned in Gazebo using a custom ROS2 launch file (`scara.launch.py`) that starts Gazebo Harmonic, publishes the robot description via `robot_state_publisher`, spawns the entity, and bridges joint states to the ROS2 topic `/joint_states`.

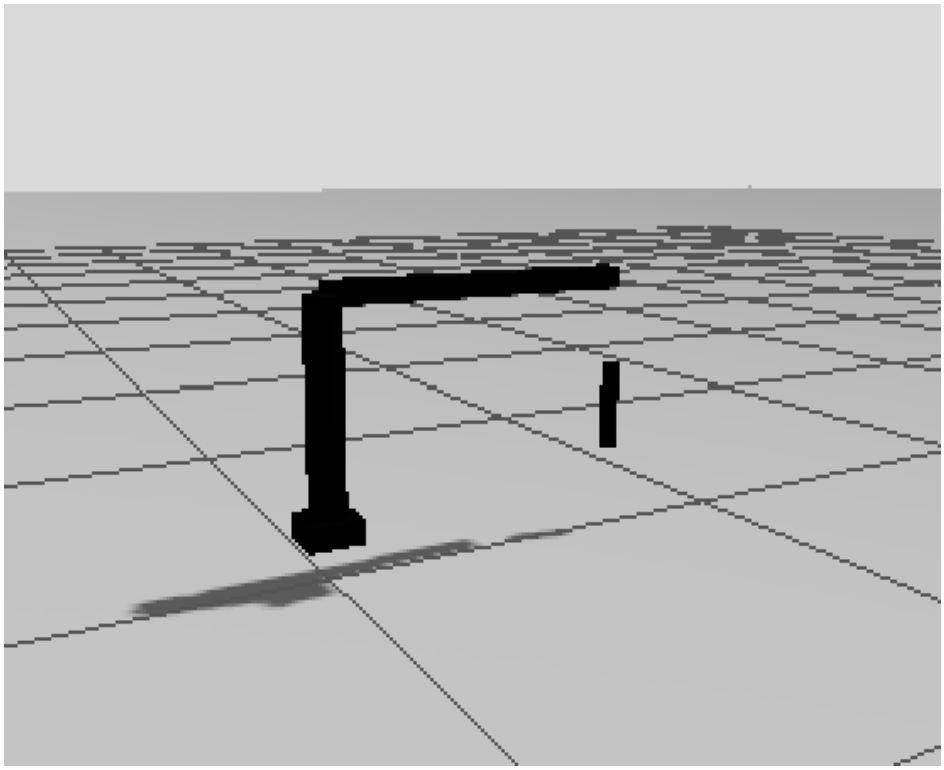

Figure 1: SCARA robot spawned in Gazebo Harmonic (ROS2 Jazzy). The base column, two horizontal arm links, and the vertical prismatic end-effector shaft are visible.

1.4 URDF Definition

The robot was defined in `scara.urdf`. Key structural elements are shown below. The full file is included with the submission.

Listing 1: URDF joint definitions (excerpt)

```
<!-- Joint 1: revolute about z -->
<joint name="joint_1" type="revolute">
    <parent link="base_link"/>
    <child link="link_1"/>
    <origin xyz="0 0 0.05"/>
    <axis xyz="0 0 1"/>
    <limit effort="1000" lower="-1.57" upper="1.57" velocity="0.5"/>
</joint>

<!-- Joint 2: revolute about z -->
<joint name="joint_2" type="revolute">
    <parent link="arm_1"/>
    <child link="link_2"/>
    <origin xyz="0.4 0 0"/>
    <axis xyz="0 0 1"/>
    <limit effort="1000" lower="-1.57" upper="1.57" velocity="0.5"/>
```

```
</joint>

<!-- Joint 3: prismatic along z -->
<joint name="joint_3" type="prismatic">
    <parent link="link_2"/>
    <child link="link_3"/>
    <origin xyz="0.3 0 0"/>
    <axis xyz="0 0 1"/>
    <limit effort="1000" lower="-0.2" upper="0.2" velocity="0.5"/>
</joint>
```

2 Forward Kinematics

2.1 Mathematical Derivation

The forward kinematics were derived using the Denavit-Hartenberg (DH) method, following the convention introduced in lecture. The total transformation from the base frame to the end-effector frame is:

$$T_3^0 = T_1^0 \cdot T_2^1 \cdot T_3^2$$

The resulting closed-form homogeneous transformation matrix is:

$$T_3^0 = \begin{bmatrix} c_{12} & -s_{12} & 0 & -a_2 s_{12} - a_1 s_1 \\ s_{12} & c_{12} & 0 & a_2 c_{12} + a_1 c_1 \\ 0 & 0 & 1 & l_1 - d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where $c_{12} = \cos(\theta_1 + \theta_2)$, $s_{12} = \sin(\theta_1 + \theta_2)$, $c_1 = \cos\theta_1$, and $s_1 = \sin\theta_1$.

The end-effector position is therefore:

$$p_x = -a_2 s_{12} - a_1 s_1 \tag{1}$$
$$p_y = \quad a_2 c_{12} + a_1 c_1 \tag{2}$$
$$p_z = l_1 - d_3 \tag{3}$$

2.2 ROS2 Node Implementation

A ROS2 Python node (`fk_node.py`) was implemented in the `scara_fk` package. The node:

1. Subscribes to `/joint_states` (published by Gazebo)
2. Computes the end-effector pose using the closed-form FK above
3. Publishes the result as a `geometry_msgs/msg/Pose` message on `/scara/ee_pose`

Listing 2: FK computation (fk_node.py excerpt)

```
def joint_state_callback(self, msg: JointState):
    theta1 = self._get_joint(msg, 'joint_1')
    theta2 = self._get_joint(msg, 'joint_2')
    d3     = self._get_joint(msg, 'joint_3')

    c1  = math.cos(theta1);  s1  = math.sin(theta1)
    c12 = math.cos(theta1 + theta2)
    s12 = math.sin(theta1 + theta2)

    px = -A2 * s12 - A1 * s1   # A1=0.40, A2=0.30
    py =  A2 * c12 + A1 * c1
    pz =  L1 - d3               # L1=0.45

    # Pure rotation about z by (theta1 + theta2) -> quaternion
    half = (theta1 + theta2) / 2.0
    pose.orientation.z = math.sin(half)
    pose.orientation.w = math.cos(half)
    self.pub.publish(pose)
```

2.3 Result

The node was run while the robot was at its default joint configuration ($\theta_1 = 0$, $\theta_2 = 0$, $d_3 = -0.2$ m). The expected end-effector position is:

$$p = \big(-0.0,\ a_1 + a_2,\ l_1 - d_3\big) = \big(0,\ 0.7,\ 0.65\big)\ \text{m}$$

This matches the terminal output shown below.

```
tracy@motherboard:~/rbe500$ ros2 topic echo /scara/ee_pose --once
A message was lost!!!
        total count change:1
        total count: 1---
A message was lost!!!
        total count change:2
        total count: 3---
A message was lost!!!
        total count change:1
        total count: 4---
A message was lost!!!
        total count change:1
        total count: 5---
A message was lost!!!
        total count change:1
        total count: 6---
position:
  x: -0.0
  y: 0.7
  z: 0.6500000000000098
orientation:
  x: 0.0
  y: 0.0
  z: 0.0
  w: 1.0
---
```

Figure 2: Terminal output of `ros2 topic echo /scara/ee_pose --once`. Position $(x, y, z) = (-0.0,\ 0.7,\ 0.65)$ m and identity orientation (all joints at zero) are reported correctly.

3 Inverse Kinematics

3.1 Mathematical Derivation

The inverse kinematics problem solves for joint values $(\theta_1, \theta_2, d_3)$ given a desired end-effector position (p_x, p_y, p_z).

Step 1 — d_3 directly from p_z:

$$d_3 = l_1 - p_z$$

Step 2 — θ_2 via the law of cosines:

$$r^2 = p_x^2 + p_y^2, \qquad \cos\theta_2 = \frac{r^2 - a_1^2 - a_2^2}{2a_1a_2}$$

$$\theta_2 = \text{atan2}\Big(+\sqrt{1-\cos^2\theta_2},\ \cos\theta_2\Big) \quad \text{(elbow-up solution)}$$

Step 3 — θ_1 algebraically:

Defining $k_1 = a_1 + a_2\cos\theta_2$ and $k_2 = a_2\sin\theta_2$, and substituting into the FK position equations yields the 2×2 linear system:

$$\theta_1 = \text{atan2}(-p_x k_1 - p_y k_2,\ \ p_y k_1 - p_x k_2)$$

3.2 ROS2 Node Implementation

A ROS2 Python node (`ik_node.py`) was implemented in the `scara_ik_node` package. The node creates a service at `/scara/compute_ik` using a custom service type `scara_ik/srv/ScaraIK`:

Listing 3: ScaraIK.srv service definition

```
# Request: desired end-effector position
float64 x
float64 y
float64 z
---
# Response: joint values and success flag
float64 theta1
float64 theta2
float64 d3
bool success
string message
```

Listing 4: IK solver (ik_node.py excerpt)

```
def ik_callback(self, request, response):
    px, py, pz = request.x, request.y, request.z

    # Step 1: d3
    d3 = L1 - pz

    # Step 2: theta2
    r_sq = px**2 + py**2
    c2   = (r_sq - A1**2 - A2**2) / (2.0 * A1 * A2)
    s2   = math.sqrt(1.0 - c2**2)          # elbow-up
    theta2 = math.atan2(s2, c2)

    # Step 3: theta1
    k1 = A1 + A2 * c2
    k2 = A2 * s2
    theta1 = math.atan2(-px*k1 - py*k2,
                        py*k1 - px*k2)

    response.theta1  = theta1
```

```
    response.theta2  = theta2
    response.d3      = d3
    response.success = True
    return response
```

3.3 Result

The IK service was tested with the `ros2 service call` command for the target position $(x, y, z) = (0.35, 0.45, 0.25)$ m.

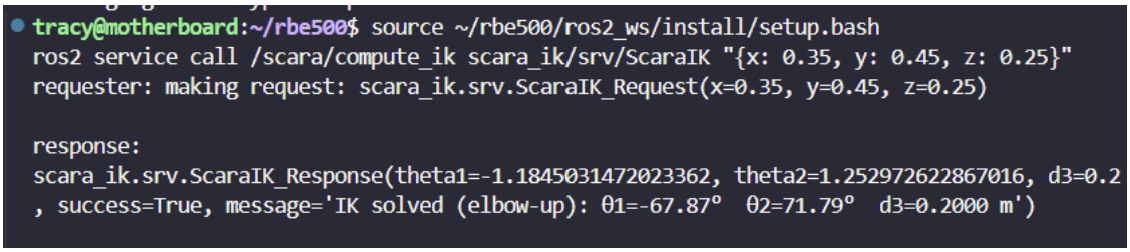

Figure 3: Terminal output of the `ros2 service call` command. The IK solver returns $\theta_1 = -67.87°$, $\theta_2 = 71.79°$, $d_3 = 0.2$ m with `success=True`.

3.4 Verification

The solution can be verified by substituting back into the FK equations:

$$\theta_1 = -67.87° = -1.1845 \text{ rad}, \quad \theta_2 = 71.79° = 1.2530 \text{ rad}, \quad d_3 = 0.2 \text{ m}$$

$$p_x = -0.30 \sin(-67.87° + 71.79°) - 0.40 \sin(-67.87°) \approx 0.35 \quad \checkmark$$

$$p_y = \quad 0.30 \cos(-67.87° + 71.79°) + 0.40 \cos(-67.87°) \approx 0.45 \quad \checkmark$$

$$p_z = 0.45 - 0.2 = 0.25 \quad \checkmark$$

4 Package Structure

The submission includes three ROS2 packages inside `ros2_ws/src/`:

- `scara_description` — URDF, launch file, and Gazebo configuration
- `scara_fk` — Forward kinematics node (Python, `ament_python`)
- `scara_ik` — Custom `ScaraIK` service definition (`ament_cmake`)
- `scara_ik_node` — Inverse kinematics service node (Python, `ament_python`)